

Arbeitsauftrag – PHP

PDO-Connection, PHP-Klasse, PHP-Schleifen, Logik mit PHP

Aufgabenbeschreibung 90 Punkte + 10 Punkte

Aufgabe 1: Erstellung der Klasse `db.php` für die Datenbankverbindung (10 Punkte)

1. Erstellen Sie eine PHP-Klasse oder PHP-Script `db.php`, die eine **Singleton-Implementierung** zur Verfügung stellt. **(freiwillig!!!!) Zusatzpunkte**
2. Die Klasse muss ein **PDO-Objekt** erzeugen, das zur Verbindung mit der Datenbank verwendet wird.
3. Stellen Sie sicher, dass:
 - a. **Exceptions** bei Verbindungsfehlern behandelt werden.

Bewertung:

- **10 Punkte:** Singleton-Pattern korrekt umgesetzt. **(Recherche notwendig)**
- 10 Punkte: PDO-Objekt wird erfolgreich erzeugt und verwendet.

Aufgabe 2: Klasse für die Buchungstabelle (30 Punkte)

1. Erstellen Sie eine PHP-Klasse/PHP-Script `Buchung.php`
Implementieren Sie mindestens **folgende Methoden** für die Buchungstabelle:
 - a. `selectAll()`: Gibt alle Buchungen zurück.
 - b. `selectByFilter(array $filter)`: Gibt Buchungen basierend auf den übergebenen Filterkriterien zurück.
 - c. `insert(array $buchung)`: Fügt eine neue Buchung hinzu.
 - d. `update(int $id, array $buchung)`: Aktualisiert eine bestehende Buchung.
 - e. `delete(int $id)`: Löscht eine Buchung anhand der ID.

Dieses Script / Diese Klasse soll für sämtliche SQL-Operationen in Bezug auf die Buchungen beinhalten.

Bewertung:

- 00 Punkte: Grundstruktur der Klasse

- 30Punkte: Funktionalität der Methoden (selectAll, insert, update, delete) wird getestet und ist korrekt.

Aufgabe 3: Dynamische Befüllung des Mockups (30 Punkte)

1. **Daten abrufen:** Verwenden Sie die Methode `selectAll()` aus der Buchung-Klasse, um die Buchungsdaten aus der Datenbank zu holen. Beim Aufruf der Startseite sollen die Buchungen ohne Filterung aufgelistet werden.
2. **Daten anzeigen:** Ersetzen Sie die statisch angelegten Buchungssätze im Bootstrap-Mockup durch die aus der Datenbank abgerufenen Daten.
3. **Schleifen und Templates:** Nutzen Sie eine Schleife, um das zurückgelieferte **Resultset** zu verarbeiten und dynamisch in das HTML-Template einzufügen.

Bewertung:

- 15 Punkte: Daten korrekt aus der Datenbank abgerufen.
- 15 Punkte: Daten dynamisch und korrekt im Mockup angezeigt.

Aufgabe 4: Integration von Filterkriterien (20 Punkte)

1. **Filterformular nutzen:** Greifen Sie auf das bereits bestehende Filterformular im Mockup zu und übergeben Sie die eingegebenen Filterkriterien an die Methode `selectByFilter(array $filter)` aus der Buchung-Klasse.
2. **Filterlogik umsetzen:** Implementieren Sie die Logik, um die Buchungen basierend auf den Filterkriterien zu selektieren (z. B. Datum, Betrag, Kategorie).
3. **Gefilterte Daten anzeigen:** Zeigen Sie die gefilterten Buchungssätze dynamisch im Mockup an.

Tipp: Sofern die Struktur des Abfrage-Ergebnisses gleich dem Ergebnis von `selectAll` ist, kann beim Aufbau der Seite mittels IF-Condition unterschieden werden.

Bewertung:

- 10 Punkte: Filterkriterien korrekt an `selectByFilter` übergeben.
- 10 Punkte: Gefilterte Buchungen korrekt im Mockup angezeigt.

Zusätzliche Hinweise

- Nutzen Sie **Prepared Statements** in Ihren Methoden, um SQL-Injection zu vermeiden.
-
- Stellen Sie sicher, dass der HTML-Code sauber und lesbar bleibt, auch wenn er dynamisch innerhalb einer Schleife generiert wird.

Bewertungsschema

- **10 Punkte:** Klasse `db.php` für die Datenbankverbindung.
- **10 Punkte:** Singleton **Zusatzpunkte**
- **30 Punkte:** Klasse `Buchung.php` mit vollständiger Methodenumsetzung.
- **30 Punkte:** Dynamische Befüllung des Mockups mit Daten aus der Datenbank.
- **20 Punkte:** Integration und Funktionalität der Filterkriterien.